Polina Khlystun

DATA SCIENCE CAMP
2026

Бібліотека Pandas

Що таке pandas? 2
Вступ 2
Series 2
DataFrame 4
Читання CSV-файлів 6
Аналіз DataFrames 7
Робота з даними 8
Очищення порожніх комірок 9
Дані неправильного формату 10
Очищення неправильних даних 11
Видалення дублікатів 11
Додаткова інформація 12
Кореляція 12
Побудова графіків 12
Список джерел 13

Що таке pandas?

Вступ

Pandas – це бібліотека Python, що використовується для аналізу даних. Дозволяє аналізувати великі дані, очищати невпорядковані набори даних, робити їх читабельними та релевантними (важливо в науці про дані).

Наука про дані (Data Science) – розділ інформатики, вивчає як зберігати, використовувати та аналізувати дані для отримання з них інформації.

Якщо Python вже встановлений, інсталювати бібліотеку можна за допомогою команди:

```
C:\User\Python>pip install pandas
```

Або:

```
C:\User\Python>python -m pip install pandas
```

Pandas зазвичай імпортують під *pd* псевдонімом. *Псевдоніми (alias) – це альтернативна назва для того самого об'єкта.* Створюється за допомогою ключового слова *as*.

```
import pandas as pd
```

Series

Серії (Series) – це одновимірний масив, що містить дані будь якого типу. Схоже на стовпець у таблиці.

Приклад коду:

```
import pandas as pd

a = [1, 7, 2]
myvar = pd.Series(a)

print(myvar)
```

Результат:

```
0    1
1    7
2    2
```

Другий стовпець містить введені дані, а перший – це індексний номер для кожного значення. Нумерація завжди починається з 0. Також за допомогою аргументу *index* можна задати власні мітки.

Приклад коду:

```
import pandas as pd

a = [1, 7, 2]
myvar = pd.Series(a, index = ["x", "y", "z"])

print(myvar)
```

Результат:

```
x    1
y    7
z    2
```

Також можна використовувати об’єкти типу «ключ/значення», як словники, при створенні серій. Ключі словника тоді стають мітками.

Приклад коду:

```
import pandas as pd

calories = {"day1": 420, "day2": 380, "day3": 390}
myvar = pd.Series(calories)

print(myvar)
```

Результат:

```
day1 420
day2 380
day3 390
```

Щоб вибрати лише деякі елементи словника, скористайтеся *index* аргументом і вкажіть лише ті елементи, які потрібно включити до серії:

```
import pandas as pd

calories = {"day1": 420, "day2": 380, "day3": 390}
myvar = pd.Series(calories, index = ["day1", "day2"])

print(myvar)
```

Результат:

```
day1 | 420
day2 | 380
```

DataFrame

Якщо серія – це стовпець, то DataFrame – це вся таблиця. Приклад коду:

```
import pandas as pd

data = {
  "calories": [420, 380, 390],
  "duration": [50, 40, 45]
}

myvar = pd.DataFrame(data)

print(myvar)
```

Результат:

```
     | calories | duration
0    | 420      | 50
1    | 380      | 40
2    | 390      | 45
```

DataFrame – це двовимірна структура даних, така як двовимірний масив або таблиця з рядками та стовпцями.

Pandas використовує *loc* атрибут для повернення одного або кількох заданих рядків. Приклад коду 1:

```
import pandas as pd

data = {
  "calories": [420, 380, 390],
  "duration": [50, 40, 45]
}
```

```
df = pd.DataFrame(data)

print(df.loc[0])
```

Результат:

```
calories | 420
duration | 50
Name: 0  |
```

Приклад коду 2:

```
print(df.loc[[0, 1]])
```

Результат:

```
     | calories | duration
0    | 420      | 50
1    | 380      | 40
```

У першому випадку loc повертає Series, тому Pandas додає службову інформацію *Name: 0* – індекс цього рядка. В другому повертається DataFrame, тому тут немає *Name*, індекси (0, 1) показуються окремою колонкою зліва.

Для того щоб перший код теж видав DataFrame, можна модифікувати код:

```
print(df.loc[[0]])
```

Результат:

```
     | calories | duration
0    | 420      | 50
```

Індекси також можна перейменовувати, як це було з серіями. Приклад 1:

```
import pandas as pd

data = {
  "calories": [420, 380, 390],
  "duration": [50, 40, 45]
}

df = pd.DataFrame(data, index = ["day1", "day2", "day3"])

print(df)
```

Результат:

```
      | calories | duration
day1  | 420      | 50
day2  | 380      | 40
day3  | 390      | 45
```

Приклад 2:

```
print(df.loc[["day2"]])
```

Результат:

```
      | calories | duration
day2  | 380      | 40
```

Читання CSV-файлів

Простий спосіб зберігання великих наборів даних – це використання CSV-файлів, дослівно файлів, розділених комами (comma separated files). Такі файли містить звичайний текст і є добре відомим форматом.

Приклад коду:

```
import pandas as pd

df = pd.read_csv('data.csv')

print(df.to_string())
```

Використовуйте to_string() для друку всього DataFrame. Без цього методу Pandas поверне лише перші 5 рядків та останні 5 рядків:

```
     | Duration | Pulse  | Maxpulse | Calories
0    | 60       | 110    | 130      | 409.1
1    | 60       | 117    | 145      | 479.0
2    | 60       | 103    | 135      | 340.0
3    | 45       | 109    | 175      | 282.4
4    | 45       | 117    | 148      | 406.0
...  | ...      | ...    | ...      | ...
164  | 60       | 105    | 140      | 290.8
165  | 60       | 110    | 145      | 300.4
166  | 60       | 115    | 145      | 310.2
167  | 75       | 120    | 150      | 320.4
168  | 75       | 125    | 150      | 330.4
[169 rows x 4 columns]
```

Максимальна кількість рядків визначена в налаштуваннях Pandas. Її можна змінити за допомогою оператора *max_rows*:

```
import pandas as pd

pd.options.display.max_rows = 9999
df = pd.read_csv('data.csv')

print(df)
```

Аналіз DataFrames

Одним з найбільш використовуваних методів для швидкого огляду DataFrames є метод *head()*. Він повертає заголовки та задану кількість рядків, починаючи зверху.

Приклад коду:

```
import pandas as pd

df = pd.read_csv('data.csv')

print(df.head(10))
```

Якщо кількість рядків не вказано, метод head() повертає 5 верхніх рядків.

Результат:

```
     Duration  Pulse  Maxpulse  Calories
0      60      110      130      409.1
1      60      117      145      479.0
2      60      103      135      340.0
3      45      109      175      282.4
4      45      117      148      406.0
```

Також існує метод *tail()* для перегляду останніх рядків DataFrame. Він повертає заголовки та задану кількість рядків, починаючи знизу.

Приклад коду:

```
import pandas as pd

df = pd.read_csv('data.csv')

print(df.tail())
```

Результат:

	Duration	Pulse	Maxpulse	Calories
164	60	105	140	290.8
165	60	110	145	300.4
166	60	115	145	310.2
167	75	120	150	320.4
168	75	125	150	330.4

Метод *info()* показує, скільки ненульових значень присутні в кожному стовпці:

Приклад коду:

```
print(df.info())
```

Результат:

	Column	Non-Null Count
0	Duration	169 non-null
1	Pulse	169 non-null
2	Maxpulse	169 non-null
3	Calories	164 non-null

Порожні значення, або значення Null, можуть погано вплинути на аналіз даних, їх варто редагувати або видаляти.

Робота з даними

В Data Science неправильними даними можуть бути:

- порожні клітинки,
- дані у неправильному форматі,
- неправильні дані,
- дублікати.

Очищення порожніх комірок

Один зі способів роботи з порожніми клітинками – це видалення рядків, які їх містять. Зазвичай це нормально, оскільки набори даних можуть бути великими і видалення кількох рядків не матиме великого впливу на результат.

Методи для роботи з порожніми клітинками

`dropna()`	Метод повертає новий DataFrame і **не змінює** оригінал
`dropna(inplace = True)`	Метод видаляє усі рядки, що містять значення NULL, з вихідного DataFrame (не повертаючи новий)
`fillna()`	Дозволяє замінити порожні клітинки на вказане значення. Наприклад: `fillna(130, inplace = True)` Або замінити порожні значення лише для одного стовпця: `fillna({"Calories": 130}, inplace=True)`

`mean()`	Обчислює середнє значення стовпця. В подальшому ним можна замінити будь-які порожні значення:
	
	`x = df["Calories"].mean()`
	`df.fillna({"Calories": x}, inplace=True)`

`median()`	Обчислює медіану стовпця.
`mode()`	Обчислює моду стовпця.

***Середнє** – сума всіх значень, поділена на кількість. **Медіана** – середній елемент впорядкованого набору. **Мода** – значення, яке з'являється найчастіше.*

Дані неправильного формату

Комірки з даними неправильного формату можуть ускладнити або навіть зробити неможливим аналіз даних. Щоб виправити це, можна видалити рядки або зробити всі клітинки в стовпці однакового формату.

Основні методи конвертації в pandas

`pd.to_datetime()`	Перетворює значення у формат дати/часу.
	
	`df['Date'] = pd.to_datetime(df['Date'], format='mixed')`
	
	Порожні значення потім можна видалити:
	
	`df.dropna(subset=['Date'], inplace = True)`

`pd.to_numeric()`	Перетворює значення у числа (int або float). Якщо стовпець містить хоч одне дробове значення або NaN – всі значення будуть float.
`astype()`	Явно змінює тип даних (int, float, string). Тобто розробник сам вказує необхідний йому тип.
`convert_dtypes()`	Автоматично підбирає найкращі типи для всіх колонок.
`infer_objects()`	Пробує уточнити типи для колонок типу object.
`.str`	Дозволяє обробляти текст (рядки).
`astype('category')`	Перетворює колонку в категоріальний тип (вроді enum).

*Пропущенні значення для типів даних: float → **NaN**, datetime → **NaT**, object → **None**.*

Очищення неправильних даних

«Неправильні дані» не обов'язково порожні клітинки чи неправильного формату, вони можуть бути просто неправильними. Наприклад якщо хтось зареєстрував «199» замість «1.99».

Такі значення можна замінити:

```
for x in df.index:
  if df.loc[x, "Duration"] > 120:
    df.loc[x, "Duration"] = 120
```

або видалити:

```
for x in df.index:
  if df.loc[x, "Duration"] > 120:
    df.drop(x, inplace = True)
```

Видалення дублікатів

Дублікати – це рядки, які були зареєстровані більше одного разу. Щоб їх виявити, використовують метод *duplicated()*. Він повертає логічні значення для кожного рядка. Тобто, якщо рядки 11 та 12 одинакові, результат буде наступним:

```
...    ...
11     False
12     True
...    ...
```

Щоб видалити дублікати, використовують метод *drop_duplicates()*:

```
df.drop_duplicates(inplace = True)
```

Варто пам'ятати, метод (inplace = True) не повертає новий DataFrame, а видаляє усі дублікати з оригінального DataFrame.

Додаткова інформація

Кореляція

Метод *corr()* обчислює силу зв'язків між числовими стовпцями у наборі даних, ігноруючи стовпці нечислового типу. Результат – це матриця кореляції, де показано, як кожен стовпець пов'язаний з іншими.

Значення змінюється від -1 до 1, де:

- 1 це ідеальна пряма залежність (обидва зростають разом, наприклад «більше годин навчання → більше балів»),
- 0.9 – 0.6 або -0.9 – -0.6 це сильний (гарний) зв'язок,
- 0.5 – 0.3 це середній зв'язок,
- 0.2 і менше це дуже слабкий або відсутній зв'язок (наприклад «розмір ноги → оцінки»),
- 0 це зв'язку немає,
- -1 це ідеальна обернена залежність (наприклад «менше годин навчання → менше балів»).

Побудова графіків

Метод *plot()* використовується для побудови графіків. Для відображення графіка використовується бібліотека *matplotlib (pyplot)*. Побудувати графік для всіх числових колонок можна наступним чином:

```
import matplotlib.pyplot as plt

df.plot()
plt.show()
```

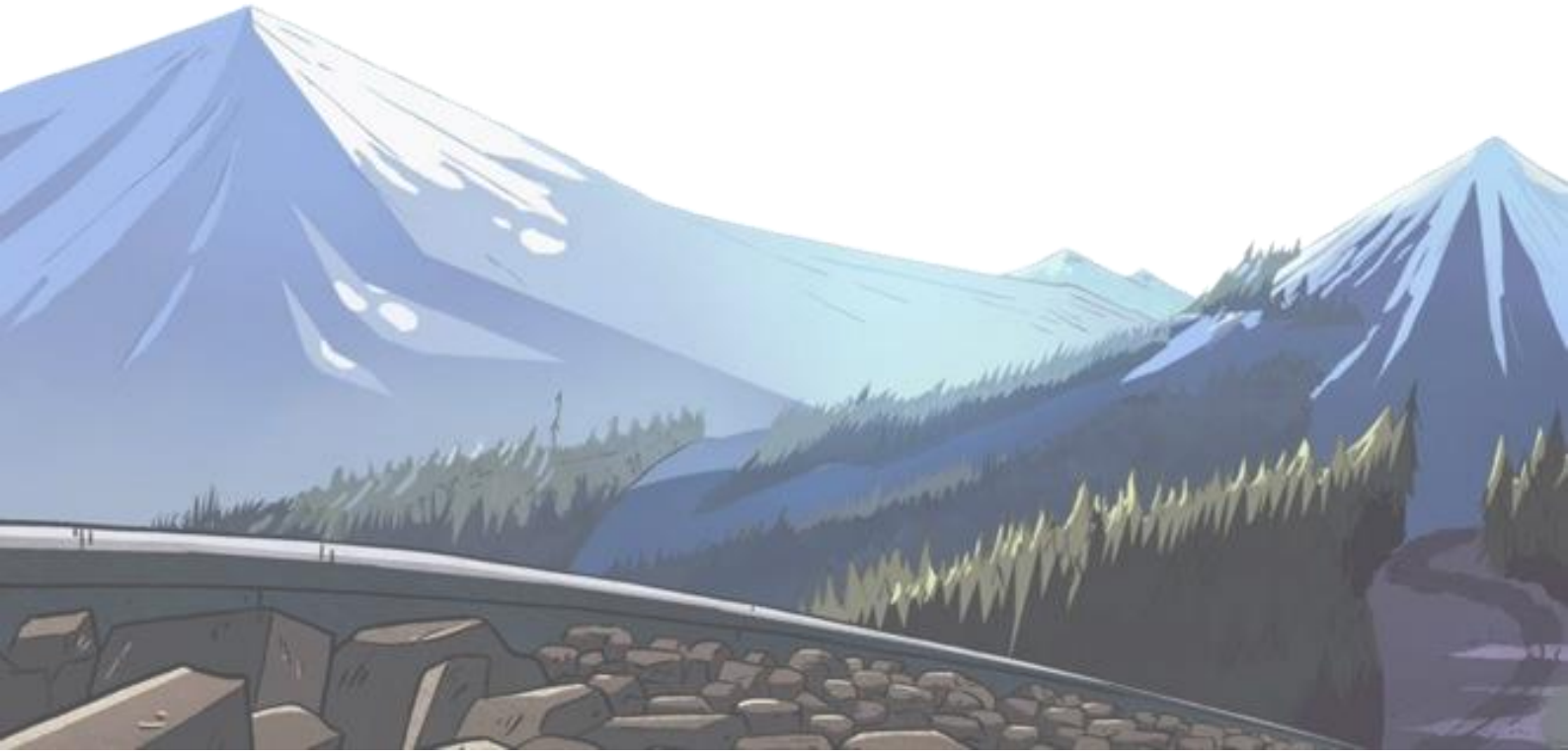
Задати тип діаграми можна за допомогою аргументу *kind*:

```
df.plot(kind = 'scatter', x = 'Duration', y = 'Calories')
```

Точковий графік (Scatter plot) – використовується для показу зв'язку між двома змінними:

- точки йдуть вгору – кореляція позитивна,
- вниз – негативна,
- хаос – зв'язку немає.

Гістограма (Histogram) показує розподіл значень в одній колонці. Тобто як часто зустрічаються окремі значення в колонці, наприклад скільки разів Duration було в діапазоні 50-60.

Список джерел

1. w3schools. URL: https://www.w3schools.com/python/pandas/default.asp